realnetworks

SAFR™

System Web API Guide

v1.3

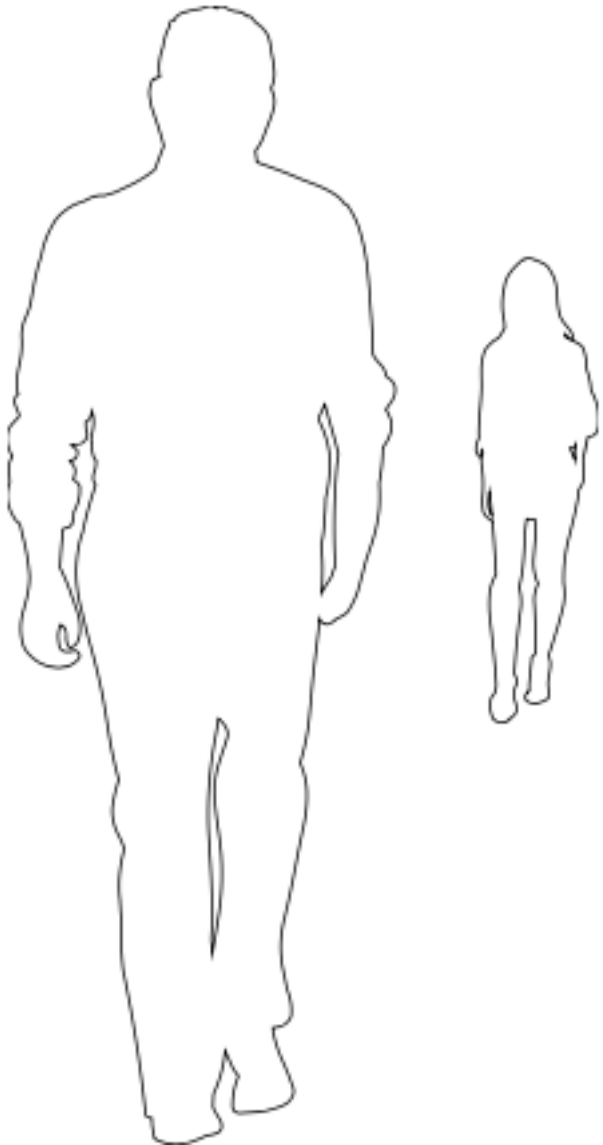
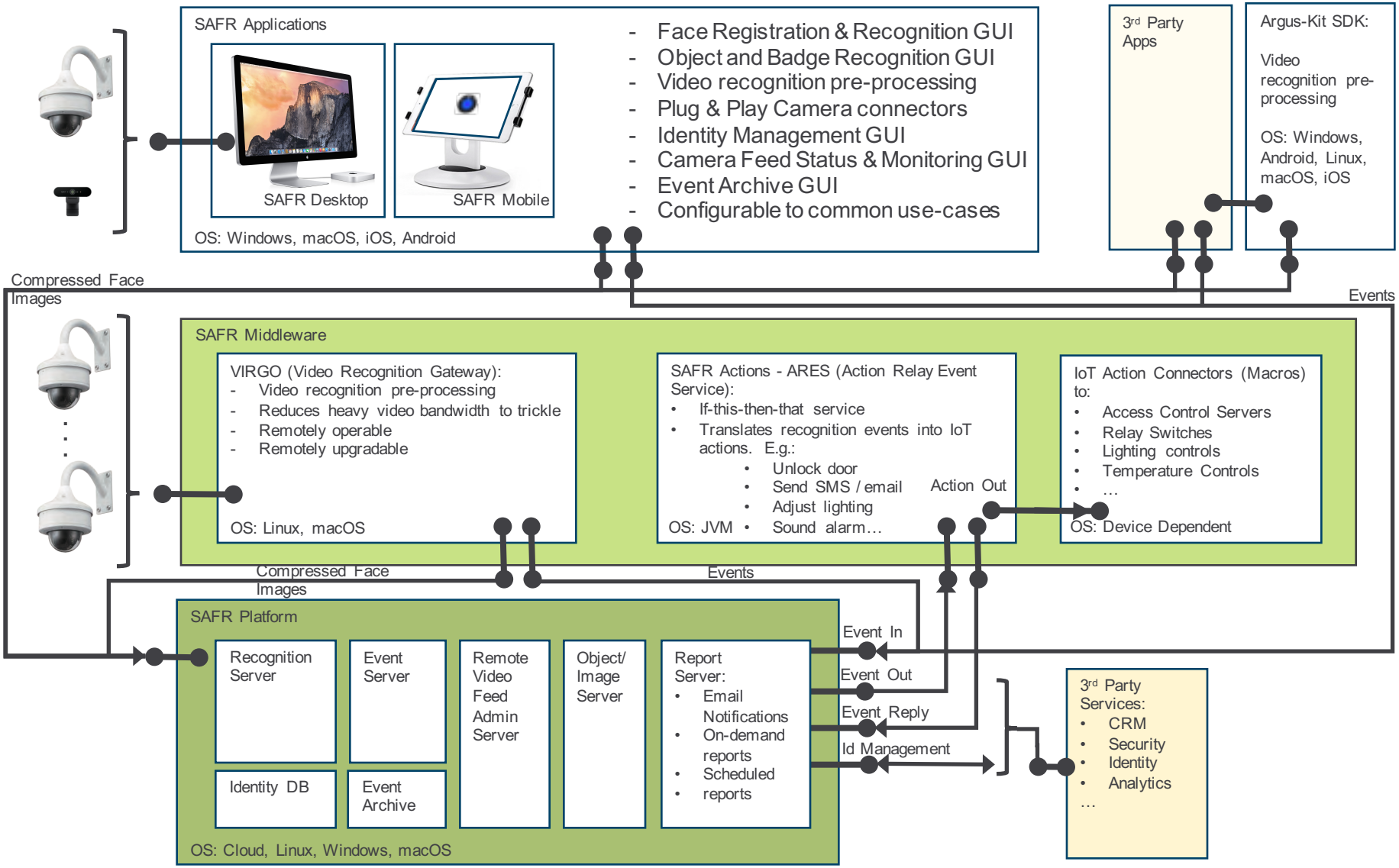
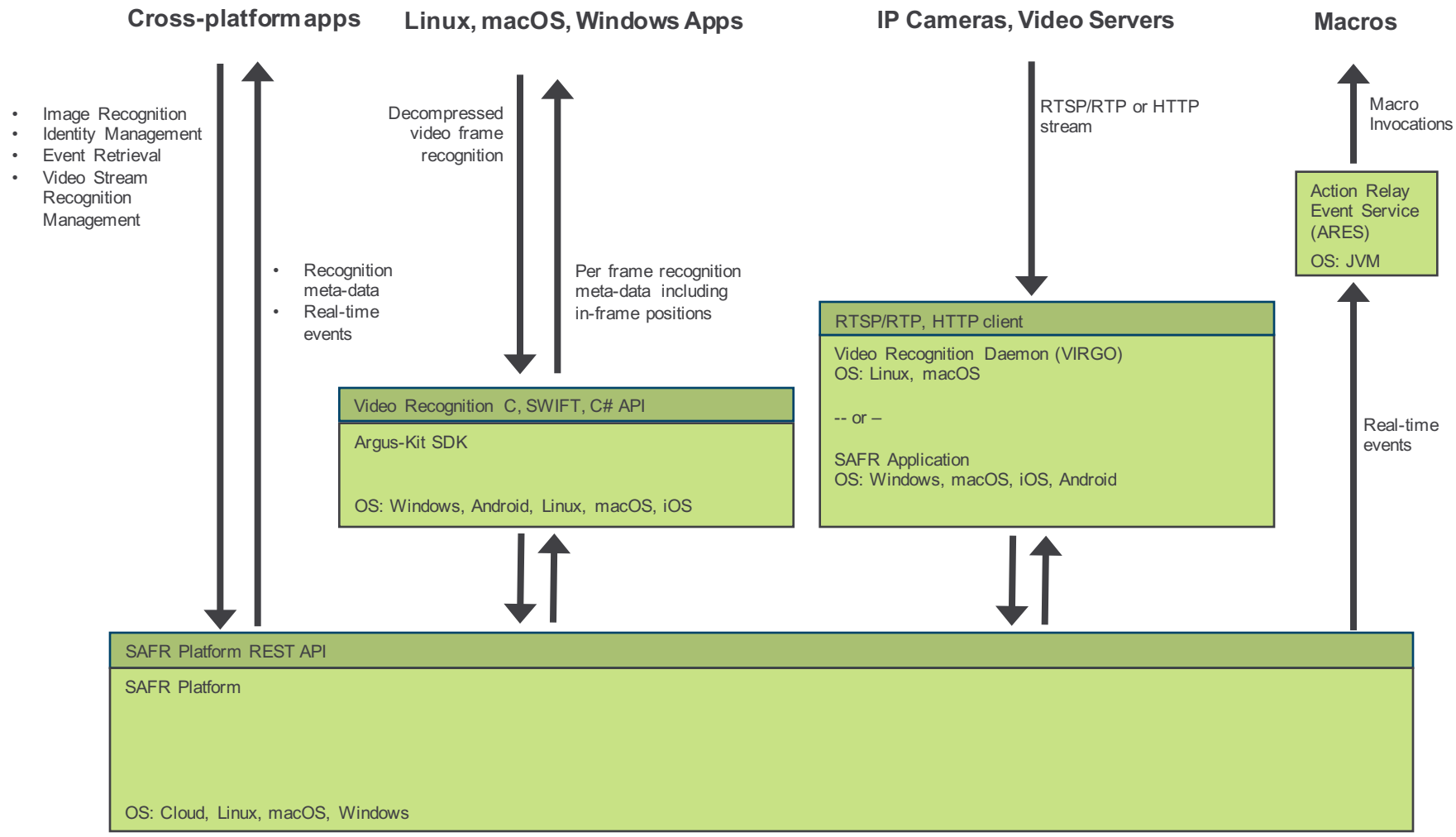


Table of Contents

SAFR Architecture



SAFR API Overview



SAFR Platform REST API

- **Table Of Contents:**
  - **Explanation of REST API End Points**
    - REST API End Points for SAFR Platform as Cloud Service
    - REST API End Points for local deployment of SAFR Platform
  - **Main Computer Vision Service API:**
    - Importing face from an image as new identity
    - Retrieving stored identities
    - Deleting stored identities
    - Retrieving images of stored identities
    - Matching images against stored identities
  - **Event Service API:**
    - Retrieving events stored in the directory
    - Retrieving images associated with the events
    - Listening for new events and retrieving them as they occur

Explanation of REST API End Points

SAFR platform is comprised of 4 services (API end-points):

- COVI - Main Computer Vision Service:
  - Identity Matching, Identity Repository, Identity Management, Detection, Recognition
- CVEV - Event Service:
  - Real-time events and event archive access
- VRGA - Video Recognition Gateway (VIRGO) Admin Service
  - VIRGO Video Feeds Status monitoring and Configuration Set-up
- CVOS - Object Service
  - Image storage and streamed meta-data between services

REST API End Points for SAFR Cloud Service

When SAFR Cloud service is used, it is offered in separate pre-production and production environments. All evaluations and experimentation work is normally performed in pre-production. Production environment is used for live deployments.

Pre-production environment has domain of int2.real.com and is referred to as **SAFR Partner Cloud**.
Production environment has domain of real.com and is referred to as **SAFR Cloud**.

SAFR Partner Cloud service has the following default end-points (this is pre-production environment):

- COVI - Main Computer Vision Service:
  API end-point: https://covi.int2.real.com/
  Documentation end-point: https://covi.int2.real.com/docs/index.html

- CVEV - Event Service:
  API end-point: https://cv-event.int2.real.com/
  Documentation end-point: https://cv-event.int2.real.com/docs/index.html

- VRGA - Video Recognition Gateway Admin Service:
  API end-point: https://virga.int2.real.com/
  Documentation end-point: https://virga.int2.real.com/docs/index.html

- CVOS - Object Service (Event related images, streamed meta-data between services):
  API end-point: https://cvos.int2.real.com/
  Documentation end-point: https://cvos.int2.real.com/docs/index.html

REST API End Points for SAFR Cloud Service
(Production Environment)

SAFR Cloud service has the following default end-points for production environment:

- COVI - Main Computer Vision Service:
  API end-point: https://covi.real.com/
  Documentation end-point: https://covi.real.com/docs/index.html

- CVEV - Event Service:
  API end-point: https://cv-event.real.com/
  Documentation end-point: https://cv-event.real.com/docs/index.html

- VRGA - Video Recognition Gateway Admin Service:
  API end-point: https://virga.real.com/
  Documentation end-point: https://virga.real.com/docs/index.html

- CVOS - Object Service (Event related images, streamed meta-data between services):
  API end-point: https://cvos.real.com/
  Documentation end-point: https://cvos.real.com/docs/index.html

REST API End Points for local deployment of SAFR Platform

SAFR Platform when hosted locally has the following default end-points. These end-points by default do not use SSL as the system is assumed to operate in secure environment. If SSL is needed, the system can be fronted with an Apache proxy and offered at odd port numbers:

- COVI - Main Computer Vision Service:
  API end-point: http://localhost:8080/covi-ws
  SSL API end-point: https://localhost:8081/covi-ws
  Documentation end-point: http://localhost:8080/covi-ws/docs/index.html

- CVEV - Event Service:
  API end-point: http://localhost:8082
  SSL API end-point: https://localhost:8083
  Documentation end-point: http://localhost:8082/docs/index.html

- VRGA - Video Recognition Gateway Admin Service:
  API end-point: http://localhost:8084
  SSL API end-point: https://localhost:8085
  Documentation end-point: http://localhost:8084/docs/index.html

- CVOS - Object Service:
  API end-point: http://localhost:8086
  SSL API end-point: https://localhost:8087
  Documentation end-point: http://localhost:8086/docs/index.html

COVI – Main Computer Vision Service API

- All of the API calls described below are on COVI service API (https://covi.int2.real.com/docs/index.html)
- All of the examples below use user identifier "userid" and user password also set to "pwd" thus containing -H "X-RPC-AUTHORIZATION: userid:pwd" for authentication purposes. Substitute userid and pwd with credentials issued for your account.
- The directory used in the examples is "main" and thus containing " -H "Authorization: main" header.
- Examples use "localhost" as the location of the API end-point. Substitute with proper IP address or domain name based on the location of the service. Refer to "REST API End Points" documentation for more information.

Importing a face from an image as a new identity:

This can be accomplished with the following call:
For local host:
curl -v -X POST -H "Content-Type:application/octet-stream" -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" -H "X-RPC-PERSON-NAME:First Last" -H "X-RPC-EXTERNAL-ID: 0000001" "http://localhost:8080/covi-ws/people?update=false" --data-binary @IMG_0000001.jpg

For SAFR Partner Cloud (pre-production environment):
curl -v -X POST -H "Content-Type:application/octet-stream" -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" -H "X-RPC-PERSON-NAME:First Last" -H "X-RPC-EXTERNAL-ID: 0000001" "**https://covi.int2.real.com**/people?update=false" --data-binary @IMG_0000001.jpg
Note: always use https when making requests over the internet

Above call will first attempt to match a face already present in the "main" directory of the account and import as new identity only if the face is not matching already existing one and only if new face meets default pose, sharpness and contrast quality. If match is found, information about matched identity will be returned (personId). If new identity is formed, returned information will include "newId" property set to true:

```
{
   "accountUpdated":  true,
   "detectionTime":  324,
   "identifiedFaces":  [
                    {
                   "personId":  "866e75a6-e22a-4077-97bb-e5dbfe1c513e",
                   "name": "First Last",
                   "newId": true,
                    ...
                    }
        ]
}
```

To force insertion of the face as new identity even when a match can be normally found, set the following HTTP header:
X-RPC-FACES-GROUPING-THRESHOLD: 0

To override pose, sharpness and contract quality criteria for import, use the following respective query parameters in the request:
min-cpq=0&min-fsq=0&min-fcq=0

Complete request with overrides in place:
curl -v -X POST -H "Content-Type:application/octet-stream" -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" -H "X-RPC-PERSON-NAME:0000001" -H "X-RPC-EXTERNAL-ID: 0000001" -H "X-RPC-FACES-GROUPING-THRESHOLD: 0" "http://localhost:8080/covi-ws/people?min-cpq=0&min-fsq=0&min-fcq=0&update=false" --data-binary @IMG_0000001.jpg

Explanation of call parameters:

-H "Authorization: main" - Indicates directory of the identities used. In this example, directory is "main". All data in different directories is separate. Use multiple directories if creating multiple identity sets for completely different uses. Maximum number of directories supported is 10 per account.

-H "X-RPC-AUTHORIZATION: userid:pwd" - Provides credentials required for API access and identifies account that will be used. Data in different accounts is entirely separate – even if the directory names in which data resides is the same.

-H "X-RPC-PERSON-NAME:First Last" - Provides person name that will associated with the inserted identity.

-H "X-RPC-EXTERNAL-ID: 0000001" - Provides external id that will associated with the inserted identity. In this case, test id of the person is being used. This external id is included in recognition responses thus allowing the identity to be correlated to information maintained external to SAFR system. Identity meta-data can also be retrieved from SAFR system by externalId as well as internal personId which are returned in insertion call response.

-H "X-RPC-FACES-GROUPING-THRESHOLD: 0" - This ensures (nearly) that newly detected face will be inserted as new identity rather than being considered the same as already present identity. Every insertion call is preceded with the check for the identity being inserted being already present in the indicated directory. If present, insertion is not made but rather already present identity is returned in response. The threshold of 0 makes almost certain that identity being inserted will be considered different from any other identity. In case of an identical image already in the directory, the face in the image would not be inserted even at threshold of 0 but be considered already matching existing identity. Attribute newId=true in the response will indicate if this call inserted the new face and thus generated new identifier:

```
"identifiedFaces": [
    {
        "personId": "866e75a6-e22a-4077-97bb-e5dbfe1c513e",
        "personExternalId": "0000001”,
        "name": "0000001”,
        "newId": true,
        ...
    }
]
```

min-cpq=0&min-fsq=0&min-fcq=0 - These parameters ensure that image insertion will not be rejected due to poor face image quality. API normally rejects insertion of face images that do not have sufficient quality (according to configurable criteria) to be used as solid reference. Above properties refer to Center Pose Quality, Face Sharpness Quality and Face Contrast Quality accordingly. Storing low quality references degrades the accuracy of the system and should be avoided if possible.

update=false - In case new face is a match to already inserted identity in the indicated directory, information for this identity will not be updated with the new face and meta-data when update is set to false. If update was set to true, last face posted would be used to update the existing matching reference (given it meets the insertion quality criteria).

Retrieving stored identities:

To retrieve all inserted identities from the directory containing less than 10000 faces, use the following API call.

curl -v -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" "http://localhost:8080/covi-ws/people"

To retrieve inserted identities from the directory in highly efficient, paged manner supporting directories of any size, use the following API call:

curl -v -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" "http://localhost:8080/covi-ws/rootpeople?start-index=0"

To retrieve already inserted identity from the directory by personId, use the following API call:

curl -v -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" "http://localhost:8080/covi-ws/people/866e75a6-e22a-4077-97bb-e5dbfe1c513e"

To retrieve already inserted identity from the directory by externalId (e.g. "0000001"), use the following API call:

curl -v -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" "http://localhost:8080/covi-ws/people/external/0000001"

Deleting stored identities:

To delete identity from the directory by personId, use the following API call:

curl -v -X DELETE -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" "http://localhost:8080/covi-ws/people/866e75a6-e22a-4077-97bb-e5dbfe1c513e?recursive=true"

The recursive=true parameter ensure all persons merged into same identity are deleted. Each merged person may be represented with different face modalities of the persons (e.g. with sun-glasses, without sun-glasses, with make-up etc…). If not specified, only specific person (face modality) will be deleted while retaining others (if they exist).

Retrieving images of stored identities – easy method:

The easiest way to retrieve face image of stored identity is by using object server GET /person/<personId>/face API (supported since SAFR version 1.4.0) and personId returned by GET /people or GET /rootpeople calls:

For local host:

```
curl -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION:
userid:pwd" "http://localhost:8086/person/<personId>/face"
```

For SAFR Partner Cloud (pre-production environment):

```
curl -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION:
userid:pwd" "https://cvos.int2.real.com/person/<personId>/face"
```

E.g.:

```
curl -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION:
userid:pwd" "https://cvos.int2.real.com/person/60046fb9-5b5d-4d2b-a3aa-6345e43da53d/face"
```

This method works for cloud and locally hosted systems for images with and without application level encryption applied to them. However, it is not as efficient as methods described next.

Retrieving images of stored identities – efficient method:

For highly efficient (most direct) retrieval of images stored for identifies, identity image URL should be leveraged.

URL to image of stored identity is returned by GET /people and GET /rootpeople calls in imageURI property. The image referenced is used to represent the identity. unmergedImageURI is provided in addition if image different from the one referenced by imageURI exists for the identity and can be used to retrieve image of the specific face modality (e.g. face image with sun-glasses) associated with matching personId.

For locally hosted system, the provided URI will be of the following form:

cvos://obj/<image-guid>

To retrieve the image, issue http:// request to the Object Server (CVOS) API end-point as follows:

curl -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" "http://localhost:8086/obj/<image-guid>"

For cloud hosted system, the provided URI can take one of the two following forms:

For images not requiring application level decryption (applicable to some early and demo accounts):
https://<path>

For application-level encrypted images:
ehttps://<path>

Images referenced via ehttps:// scheme will have to be decrypted after download before they can be viewed. The encrypted image data can be downloaded by using https:// protocol scheme with the same path:
https://<path>

The decryption is described next.

Retrieving images of stored identities – decryption of application level encrypted images:

Images referenced via ehttps:// scheme (applicable to cloud accounts only when efficient image retrieval method is used) will have to be decrypted after download before they can be viewed. The encrypted image data can be downloaded by using https:// protocol scheme with the same path provided in ehttps:// URL:
https://<path>

To decrypt the downloaded image, the following steps need to be performed.

1.) Account specific decryption key must be retrieved via the following API call:
curl -X GET -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userid:pwd" "https://covi.int2.real.com/obj/imagekey"

Response contains JSON formatted base64 form of the key:

```
{
   "key": "<base64_encoded_key>"
}
```

E.g.:

```
{
   "key": "yJgLFSH/Ypsb1wycx2TzZnvTwCob4KZHrclYgqZxrz0="
}
```

2.) To decrypt the image, 16 byte IV prefix needs to be extracted from the encrypted image data and the rest of the data decrypted using the IV and the decryption key.

```
IV = first_16_bytes_of_encrypted_image_data
Key = base64Decode(base64_encoded_key)
EncryptedImageBody = encrypted_image_data_starting_with_byte_17
DecryptedImage = getCipher(AES/CBC/PKCSPadding).decode(IV, Key, EncryptedImageBody)
```

E.g.:
openssl dec -aes-256-cbc -in EncryptedImageBody.enc -out ImageBody.jpg -K $Key -iv $IV

Matching images against stored identities:

To perform recognition in an image against the identities inserted into a directory use the following call. To make sure nothing in the directory is inserted or updated, use insert=false&update=false query parameters:

curl -v -X POST -H "Content-Type:application/octet-stream" -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userId:pwd" "http://localhost:8080/covi-ws/people?insert=false&update=false" --data-binary @IMG_0001001.jpg

To get top 5 matches with probabilities of match, use similar_limit query parameter in POST /people call:

curl -v -X POST -H "Content-Type:application/octet-stream" -H "Authorization: main" -H "X-RPC-AUTHORIZATION: userId:pwd" "http://localhost:8080/covi-ws/people?insert=false&update=false&similar_limit=5" --data-binary @IMG_0001001.jpg

Above call will return top 5 matches for each face found in the passed in image. The order in which found faces (identities) are returned is arbitrary (not sorted by similartyScore). For each found face, top 5 matching identities will be provided in order of most similar first (highest similartyScore first).

The probability of a match is returned in "similarityScore" attribute of records returned in similar array. The score of 1.0 (100%) is tied to the set X-RPC-FACES-GROUPING-THRESHOLD . Thus faces matching exactly at the threshold will have similarityScore of 1.0, those matching to greater extent score > 1.0, and those matching to lesser extent score < 1.0 . The lower bound is 0 and upper bound is 2 / (2 – sqrt(X-RPC-FACES-GROUPING-THRESHOLD)) . For default X-RPC-FACES-GROUPING-THRESHOLD setting of 0.54, upper bound is 1.581 . Identity returned outside of the similar array is best match with similarityScore >= 1.0 (if any).

Details on the response to this API call follow.

Top 5 matches response:


```
{
  "accountUpdated": false,
  "detectionTime": 616,
  "identifiedFaces": [
    {
      "attributes": {
        "centerPoseQuality": 0.58317417,
        "confidence": 0.99978894,          <— this is face detection confidence
                                             DO NOT USE MATCH PROBABILITY
        "contrastQuality": 0.61625,
        "detectionVersion": 2,
        "dimension": {
          "height": 200,
          "width": 164
        },
        "landmarks": {
          "left-eye-center": {
            "x": 0.31614387,
            "y": 0.38013837
          },
          "left-mouth-corner": {
            "x": 0.34892383,
            "y": 0.7837046
          },
          "nose-tip": {
            "x": 0.55630475,
            "y": 0.6482113
          },
          "right-eye-center": {
            "x": 0.7781885,
            "y": 0.37358207
          },
          "right-mouth-corner": {
            "x": 0.74510413,
            "y": 0.7801639
          }
        },
        "provider": "RCV",
        "sharpnessQuality": 0.6402198
      },
      "lastOccurrenceDate": 1521757646850,
      "mediaId": "a058d139-4429-4e3f-8603-38290acd3326",
      "occurrence": 2,
      "offsetX": 0.23326,
      "offsetY": 0.36203,
      "personId": "60046fb9-5b5d-4d2b-a3aa-6345e43da53d",     <—— this is best match
                                                               identity with similarityScore >= 1.0 (if any)
      "relativeHeight": 0.10982,
      "relativeWidth": 0.06004,
      "rootPersonAddDate": 1521747721069,
      "similar": [
        {
          "ignore": false,
          "lastOccurrenceDate": 1521747721069,
          "occurrence": 1,
          "personId": "60046fb9-5b5d-4d2b-a3aa-6345e43da53d",
          "similarityScore": 1.3789116          ----> best match has probability of 1.3789116
                                                      thus also listed above since >= 1.0
        },
        {
          "ignore": false,
          "lastOccurrenceDate": 1521747721069,
          "occurrence": 1,
          "personId": "328a6c14-c4b6-483b-8163-ac7390078a3f",
          "similarityScore": 0.6425859
        },
        {
          "ignore": false,
          "lastOccurrenceDate": 1521747721069,
          "occurrence": 1,
          "personId": "48328c18-044d-41d8-93c6-73b7e6b68297",
          "similarityScore": 0.5377595
        },
        {
          "ignore": false,
          "lastOccurrenceDate": 1521747721069,
          "occurrence": 2,
          "personId": "83b86722-99d2-4327-8ded-a21be3d2382c",
          "similarityScore": 0.42808503
        },
        {
          "ignore": false,
          "lastOccurrenceDate": 1521723923106,
          "occurrence": 1,
          "personId": "ae4c5292-1e3a-4495-ba9a-6bd3d6c371e2",
          "similarityScore": 0.4004748
        }
      ],
      "tags": []
    }
  ]
}
```

Explanation of top 5 matches response:

- API response returns one entry in identifiedFaces array for each detected face:

```
"identifiedFaces": [
        { ... },
        { … },
        { … },
                ...
]
```

  - The order in which found faces (identities) are returned is arbitrary (not sorted by similartyScore).

- Only faces that positively identified (recognized with similarityScore >= 1.0) are given person id:

```
"identifiedFaces": [
        {
          "personId": "866e75a6-e22a-4077-97bb-e5dbfe1c513e",
          "personExternalId": " 0000001”,
          "name": " 0000001”,
          ...
        },
        { … },
        { … },
              ...
]
```

Notes:

- Positive identification is governed by X-RPC-FACES-GROUPING-THRESHOLD. To override default setting, X-RPC-FACES-GROUPING-THRESHOLD header can be passed in each call.
- Threshold setting guidelines:
  - **0.54 – Secure Access (default)**
    Very high confidence identification with very low false positives
  - **0.67 – Traffic monitoring**
  - **0.84 – Celebrity matching**
  - **0.94 – Matching the list of interest in very small, grainy, blurry images or video.**

- Each detected face contains 5 most similar identities since the API call requested top-5 matches (similar_limit=5). Each similar identity has id (personId and if set externalId) and similarityScore. The identities in similar array are sorted by similarityScore in descending order (highest similarityScore first):


```
"identifiedFaces": [
        {
          …
          “similar” : [
                    {
                "personId": "866e75a6-e22a-4077-97bb-e5dbfe1c513e",
                "name": " 0000001 ",
                "externalId": " 0000001 ",
                "similarityScore": 1.0804045,
                ...
                    },
                    {
                "personId": "92ca0413-4e67-4981-a649-cffa29c6d56c",
                "name": " 0000023 ",
                "externalId": " 0000023 ",
                "similarityScore": 0.62635994,
                ...
                    },
                    ...
          ]
        },
        {
          …
          “similar” : [
                ...
          ]
        },
        ...
]
```

- Each detected face has attributes detailing positioning, face landmarks, face image quality, CV algorithm provider (“RCV”) and face detection confidence. None of this is relevant for identification.


```
"identifiedFaces": [
                    {
                      ...
                      "attributes": {
                                    provider": "RCV",
                                                ...
                      }
                    },
                    {
                      ...
                      "attributes": {
                                    "provider": "RCV",
                                                ...
                      }
                    },
                    {
                      ...
                      "attributes": {
                                    "provider": "RCV",
                                                ...
                      }
                    }
]
```

CVEV – Event Service API

- All of the API calls described below are on CVEV service API (https://cv-event.int2.real.com/docs/index.html).
- All of the examples below use user identifier "userid" and user password also set to "pwd" thus containing -H "X-RPC-AUTHORIZATION: userid:pwd" for authentication purposes. Substitute userid and pwd with credentials issued for your account.
- The directory used in the examples is "main" and thus containing " -H "Authorization: main" header.
- Examples use "localhost" as the location of the API end-point. Substitute with proper IP address or domain name based on the location of the service. Refer to "REST API End Points" documentation for more information.

Retrieving events stored in the directory:

To retrieve all events recorded in a directory, use the following call:

For local host:
curl -X GET "http://localhost:8082/events?sinceTime=0" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main"

For SAFR Partner Cloud (pre-production environment):
curl -X GET "https://cv-event.int2.real.com/events?sinceTime=0" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main"
Note: always use https when making requests over the internet

To retrieve events currently in progress (not yet ended):

curl -X GET "http://localhost:8082/events" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main"

To retrieve any events that occurred in last 60 seconds:

curl -X GET "http://localhost:8082/events?sinceTime=<currentEpochTimeInMs-60000>" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main"

Note: Replace =<currentEpochTimeInMs-60000> with current epoch time in milliseconds subtracted by 60000 .

Retrieving images associated with the events:

Images associated with events (if configured to be recorded) can be retrieved from the object server using the event id.

To retrieve face image that triggered the event use he following call:

For local host:

curl -v -X GET "http://localhost:8086/obj/<base64(event_id)>/face" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main" > face.jpg

For SAFR Partner Cloud (pre-production environment):

curl -v -X GET "https://cvos.int2.real.com/obj/<base64(event_id)>/face" -H "X-RPC-AUTHORIZATION: userId:pwd=" -H "Authorization: main" > face.jpg

Note: always use https when making requests over the internet

In calls above replace with base64 form of event id guid for which image is needed.
For example, event guid value of D4565B3D-D5C2-4F02-AD81-49DE55AAEFF1, should be replaced with RDQ1NjVCM0QtRDVDMi00RjAyLUFEODEtNDlERTU1QUFFRkYx .

E.g.:

- echo -n D4565B3D-D5C2-4F02-AD81-49DE55AAEFF1 | base64

RDQ1NjVCM0QtRDVDMi00RjAyLUFEODEtNDlERTU1QUFFRkYx

To retrieve scene thumbnail associated with the event (if recorded) issue the following call:

curl -v -X GET "http://localhost:8086/obj/<base64(event_id)>/sceneThumb" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main" > scene.jpg

Listening for new events and retrieving them as they occur:

To listen for new events as well as modifications to prior events, issue the following event status long-poll:

curl -X GET "http://localhost:8082/event/status?since=<currentEpochTimeInMs>" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main"

Note: Replace =<currentEpochTimeInMs> with current epoch time in milliseconds

Above call will block until a new event is recorded (started) or an existing event is updated. Previously recorded but still active events (without end date) will undergo updates (e.g. person identity being assigned or event simply being given end time as face disappears from the view).

If above call times-out or returns HTTP 204, it should simply be called again.

Once above call returns 200, it will include the following information:

```
{
        lastModDate integer     - Epoch time in milliseconds of last inserted or updated event
        serverDate integer      - Epoch time on server in milliseconds.
}
```

On 200 response, record returned lasModDate and retrieve all events newly added or updated:

curl -X GET "http://localhost:8082/events?sinceModDate=<epochTimeInMsUsedInStatusCallAbove>" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main"

Note: Replace <epochTimeInMsUsedInStatusCallAbove > with value used in since parameter of /event/status call described above.

Once the events are retrieved, listen for more events by issuing /event/status call again but now using value of recorded lastModDate for value of since parameter and continue to repeat alternating retrieval and listening steps.

curl -X GET "http://localhost:8082/event/status?since=<lastModDate>" -H "X-RPC-AUTHORIZATION: userId:pwd" -H "Authorization: main"